

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-a43b6868fc6d7ddd3.jsonl`
- SHA-256: `fbfcf003380eb5a9bf269042ebac2294e89c880adf5f659a35b011185acf0dfd`
- Source modified: `2026-06-12T10:46:23+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:43:40.309Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINES.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refute if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [low] is_vfr false-positives on common CFR camera files where r_frame_rate is the tick/field rate (e.g. r=120000/1001 vs avg=60000/1001) (C:\Users\avidu\Projects\sports-data-collector\src\core\proxy.py:144-153)
r_frame_rate is ffmpeg's guessed base rate and is frequently 2x avg_frame_rate on perfectly CFR H.264 files (SPS timing carrying field/tick rate, some GoPro/Sony profiles, MKV remuxes).
 $\text{abs}(119.88-59.94)/59.94 = 1.0 \gg 0.001$, so such a file is refused as 'looks variable-frame-rate' even though frame/fps recompute is exact for it. The only remedy offered is --allow-vfr, whose help/error text then wrongly tells the operator their timestamps are 'approximate' ? and habitual --allow-vfr use erodes the gate for genuinely VFR phone footage, which is exactly the case it exists to catch. (With --allow-vfr a true VFR source does pass parity by design ? count/avg-fps/duration are preserved ? so the gate is the ONLY defense; weakening it matters.)
Proposed fix: Treat r ? 2xavg (and r ? avg) as CFR, or replace the heuristic with a packet-timestamp uniformity check (ffprobe -show_entries packet=duration_ts on the first few hundred packets).

2. user (2026-06-12T10:43:48.247Z)

```
feat/prep-annotation-proxy
fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002)
4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)
a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)
```

3. user (2026-06-12T10:43:53.398Z)

```
1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the  $\pm 0.3$  s boundary bar.
7      The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8      downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9      so frame N on the proxy IS frame N on the original and returned labels apply to
10     the original unchanged.
11
12     The ADR-002 proxy contract enforced here:
13     - video timestamps pass through untouched (`-fps_mode passthrough`, never
14       ``-r``): same frame count, same fps, no trim, no leading padding;
15     - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16       failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17       golden labels;
18     - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19       survives the handoff (the indexer keys videos by file MD5; downstream
20       processing must keep running on the ORIGINAL file).
21     """
22     import csv
23     import hashlib
24     import os
25     import shutil
26     import subprocess
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional
29
30
31     class ProxyError(RuntimeError):
32         """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35     #: Columns of the provenance manifest (one row appended per generated proxy).
36     MANIFEST_FIELDS = [
37         "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38         "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39         "encoder_settings", "created_at",
40     ]
41
42     # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43     # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44     # back-stepping; CRF 19 keeps the ball/shuttle legible.
45     DEFAULT_HEIGHT = 1080
46     DEFAULT_CRF = 19
47     DEFAULT_GOP = 15
48     DEFAULT_PRESET = "medium"
49
50
51     @dataclass
52     class ProbeInfo:
53         """The frame?time identity of one video stream, as ffprobe reports it."""
54         width: Optional[int] = None
55         height: Optional[int] = None
56         avg_fps: Optional[float] = None      # avg_frame_rate (frames / duration)
57         r_fps: Optional[float] = None        # r_frame_rate (the container's base rate)
58         nb_frames: Optional[int] = None      # demuxed packet count == frame count
59         duration: Optional[float] = None     # video stream duration, else container
60
61         @property
62         def resolution(self) -> str:
63             if self.width and self.height:
```

```

64         return f"{self.width}x{self.height}"
65     return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffmpeg rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None
72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:
77                 return None
78             return num_f / den_f
79         return float(value)
80     except ValueError:
81         return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffmpeg (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffmpeg must stop it, not degrade it.
91     """
92     if not shutil.which("ffmpeg"):
93         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffmpeg", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106         ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffmpeg failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:
117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)
125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's

```

```

129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),
137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )
142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                        crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                        preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")
190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193 def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:

```

```

194         """Check the proxy carries the source's exact frame?time identity.
195
196     Returns a list of human-readable failures (empty == parity holds). Frame
197     count must match EXACTLY; fps within 0.1%; duration within a small slack
198     (AAC priming can pad the container by a few hundredths of a second) that
199     still catches any real trim.
200     """
201     errors = []
202
203     if src.nb_frames is None or proxy.nb_frames is None:
204         errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
205     elif src.nb_frames != proxy.nb_frames:
206         errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208     if src.avg_fps is None or proxy.avg_fps is None:
209         errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210     elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211         errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213     if src.duration is None or proxy.duration is None:
214         errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215     else:
216         tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217         if abs(proxy.duration - src.duration) > tol:
218             errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                           f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221     return errors
222
223
224 def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225     """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226     digest = hashlib.md5()
227     with open(path, "rb") as f:
228         for chunk in iter(lambda: f.read(chunk_size), b""):
229             digest.update(chunk)
230     return digest.hexdigest()
231
232
233 def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234     """Append one provenance row, writing the header only on first creation."""
235     parent = os.path.dirname(os.path.abspath(manifest_path))
236     os.makedirs(parent, exist_ok=True)
237     is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238     with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239         writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240         if is_new:
241             writer.writeheader()
242         writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

4. assistant (2026-06-12T10:43:58.856Z)

Now let me see how `is\_vfr` is used in the CLI and what the error/help text says.

5. user (2026-06-12T10:44:00.209Z)

```

docs\DECISIONS.md:116: durations match (also surfaces VFR sources, where `frame / fps` is
already unsafe).
src\cli\curate.py:686:         if proxylib.is_vfr(src_info) and not args.allow_vfr:
src\cli\curate.py:690:             "exact for constant-frame-rate sources. Re-run with

```

```

--allow-vfr to "
src\cli\curate.py:1159:     prep_parser.add_argument("--allow-vfr", action="store_true",
tests\test_proxy.py:19:     build_proxy_command, is_vfr, md5_file, parse_rate, probe_video,
tests\test_proxy.py:62: def test_is_vfr():
tests\test_proxy.py:64:     assert not is_vfr(cfr)
tests\test_proxy.py:65:     vfr = ProbeInfo(avg_fps=59.2, r_fps=60.0)
tests\test_proxy.py:66:     assert is_vfr(vfr)
tests\test_proxy.py:67:     # Unknown rates are treated as VFR (conservative: force the explicit
flag).
tests\test_proxy.py:68:     assert is_vfr(ProbeInfo(avg_fps=None, r_fps=30.0))
tests\test_proxy.py:69:     assert is_vfr(ProbeInfo(avg_fps=30.0, r_fps=None))
tests\test_proxy.py:197:         allow_vfr=False, force=False,
tests\test_proxy.py:255: def test_prep_proxy_refuses_vfr_without_flag(cli, tmp_path):
tests\test_proxy.py:258:     vfr_info = ProbeInfo(width=3840, height=2160, avg_fps=59.2,
r_fps=60.0,
tests\test_proxy.py:260:     with mock.patch("src.core.proxy.probe_video",
return_value=vfr_info):
tests\test_proxy.py:267: def test_prep_proxy_allow_vfr_proceeds(cli, tmp_path):
tests\test_proxy.py:268:     args = _prep_args(tmp_path, allow_vfr=True)
tests\test_proxy.py:270:     vfr = ProbeInfo(width=3840, height=2160, avg_fps=59.2, r_fps=60.0,
tests\test_proxy.py:272:     vfr_proxy = ProbeInfo(width=1920, height=1080, avg_fps=59.2,
r_fps=60.0,
tests\test_proxy.py:279:     with mock.patch("src.core.proxy.probe_video", side_effect=[vfr,
vfr_proxy]), \
src\core\proxy.py:144: def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:

```

6. user (2026-06-12T10:44:04.836Z)

```

650         scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
651         but with the source's EXACT frame count / fps / timeline, so frame N
652         on the proxy is frame N on the original and returned labels apply to
653         the original unchanged. Parity is verified after the encode; a proxy
654         that fails verification is deleted, never shared. Provenance (source +
655         proxy MD5s) is appended to the proxy manifest ? downstream processing
656         (import-local, export, the indexer) keeps running on the ORIGINAL file.
657         """
658         from src.core import proxy as proxylib
659
660         src = args.video_path
661         if not os.path.exists(src):
662             print(f"Error: video '{src}' not found.")
663             sys.exit(1)
664
665         stem = os.path.basename(src).split('.')[0]
666         # Mirror import-local's video_id derivation so the manifest row and a
667         # later DB registration of the same source agree on the key.
668         video_id = args.video_id or stem.lower().replace(" ", "_")
669         out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
670         if os.path.abspath(out) == os.path.abspath(src):
671             print("Error: --out must differ from --video-path.")
672             sys.exit(1)
673         if os.path.exists(out) and not args.force:
674             print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
675             sys.exit(1)
676
677         try:
678             src_info = proxylib.probe_video(src)
679         except proxylib.ProxyError as e:
680             print(f"Error: {e}")
681             sys.exit(1)
682         if src_info.avg_fps is None:
683             print(f"Error: could not determine fps of '{src}' ? the frame<->time "
684                   f"contract needs it.")
685             sys.exit(1)
686         if proxylib.is_vfr(src_info) and not args.allow_vfr:

```

```

687         print(f"Error: '{src}' looks variable-frame-rate "
688               f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
689         print(" The vendor pipeline recomputes times as frame/fps, which is only "
690               "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
691               "proceed anyway (timestamps stay identical to the original, but "
692               "frame/fps seconds are approximate for BOTH files).")
693     sys.exit(1)
694
695     os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
696     cmd = proxylib.build_proxy_command(src, out, height=args.height,
697                                       crf=args.crf, gop=args.gop,
698                                       preset=args.preset)
699     print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
700           f"fps {src_info.avg_fps:g} preserved)...")
701     try:
702         proxylib.run_ffmpeg(cmd)
703         proxy_info = proxylib.probe_video(out)
704     except proxylib.ProxyError as e:
705         print(f"Error: {e}")
706         sys.exit(1)
707
708     errors = proxylib.verify_parity(src_info, proxy_info)
709     if errors:
710         print("Error: proxy FAILED frame<->time parity verification:")
711         for err in errors:
712             print(f" - {err}")
713         try:
714             os.remove(out)
715             print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
716                   f"'poisons golden labels').")
717         except OSError:
718             print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
719                   f"share it with the vendor.")
720         sys.exit(1)
721
722     print("Hashing source + proxy (provenance)...")
723     source_md5 = proxylib.md5_file(src)
724     proxy_md5 = proxylib.md5_file(out)
725
726     from datetime import datetime, timezone
727     proxylib.append_manifest_row(args.manifest, {
728         "video_id": video_id,
729         "source_path": os.path.abspath(src),
730         "source_md5": source_md5,
731         "proxy_path": os.path.abspath(out),
732         "proxy_md5": proxy_md5,
733         "fps": f"{src_info.avg_fps:g}",
734         "frame_count": src_info.nb_frames,
735         "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None
736     else "",
737         "source_resolution": src_info.resolution,
738         "proxy_resolution": proxy_info.resolution,
739         "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
740                             f"preset={args.preset} height<={args.height} "
741                             f"fps_mode=passthrough audio=aac160k"),
742         "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
743     })
744
745     src_mb = os.path.getsize(src) / (1024 * 1024)
746     out_mb = os.path.getsize(out) / (1024 * 1024)
747     print("\n=== Annotation proxy ready ===")
748     print(f" Video ID      : {video_id}")
749     print(f" Source         : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5
750 {source_md5})")
751     print(f" Proxy          : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5

```

```

{proxy_md5}))")
750         print(f" Parity      : VERIFIED ? {src_info.nb_frames} frames @
{src_info.avg_fps:g} fps on both")
751         print(f" Manifest    : {args.manifest}")
752         print("\nNext:")
753         print(" 1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
754             "box of every rally on the exact contact/dead frame).")
755         print(" 2. On delivery, convert-cvat may use --video-path with either file "
756             "(identical fps/frame count by construction).")
757         print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
758             "indexer keys videos by its MD5.\n")
759
760     def convert_cvat(self, args):
761         """Convert a vendor CVAT rally export into our canonical per-rally CSV.
762
763         Absorbs the CVAT bounding-box delivery format (rally fields carried as box
764         attributes) on the ingestion side: each rally's time is recomputed from
765         its frame range at the video's true fps, and content defects (overlaps,
766         zero-duration, off-vocabulary endings, coarse sampling) are written to an
767         issues report rather than silently "fixed". The emitted CSV imports via
768         `import-local`; any blocker issue must be resolved first, since the
769         importer's validator rejects overlapping / zero-duration rallies.

```

7. assistant (2026-06-12T10:45:22.208Z)

Let me check the tests and the argparse help text, plus the ADR for intended scope.

8. user (2026-06-12T10:45:23.613Z)

```

1130                                     help="Retries per video when a download fails or returns a
low format (default: 2)")
1131     fetch_parser.add_argument("--retry-backoff", type=float, default=20.0,
metavar="SECONDS",
1132                                     help="Base backoff between retries; grows linearly per
attempt (default: 20)")
1133
1134     # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
1135     prep_parser = subparsers.add_parser(
1136         "prep-annotation-proxy",
1137         help="Re-encode a video into a light, frame-identical annotation proxy for "
1138             "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "
1139             "count/fps/timeline, parity-verified, provenance recorded")
1140     prep_parser.add_argument("--video-path", required=True,
1141                             help="The ORIGINAL source video (stays the system-of-record
file)")
1142     prep_parser.add_argument("--out",
1143                             help="Proxy output path (default:
data/videos/proxies/<stem>_proxy.mp4)")
1144     prep_parser.add_argument("--video-id",
1145                             help="Manifest key (default: derived from the filename,
like import-local)")
1146     prep_parser.add_argument("--height", type=int, default=1080,
1147                             help="Max proxy height; never upscales (default: 1080 ? "
1148                                 "720 risks losing the table-tennis ball)")
1149     prep_parser.add_argument("--crf", type=int, default=19,
1150                             help="x264 quality, lower=better (default: 19)")
1151     prep_parser.add_argument("--gop", type=int, default=15,
1152                             help="Max keyframe interval in frames; small = snappy "
1153                                 "frame-stepping (default: 15)")
1154     prep_parser.add_argument("--preset", default="medium",
1155                             help="x264 speed/size preset (default: medium)")
1156     prep_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
1157                             help="Provenance CSV to append to (default:
data/annotation_proxy_manifest.csv; "

```

```

1158             "commit it ? it is the proxy<->original mapping of
record)")
1159     prep_parser.add_argument("--allow-vfr", action="store_true",
1160                             help="Proceed on a variable-frame-rate source (frame/fps "
1161                                 "seconds are approximate; default: refuse)")
1162     prep_parser.add_argument("--force", action="store_true",
1163                             help="Overwrite an existing proxy file")
1164
1165     # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
1166     cvat_parser = subparsers.add_parser(
1167         "convert-cvat",
1168         help="Convert a vendor CVAT rally export (bounding-box XML) into the canonical
per-rally CSV")
1169     cvat_parser.add_argument("--input", required=True,
1170                             help="Path to the CVAT XML export (image- or track-mode)")
1171     cvat_parser.add_argument("--out", required=True,
1172                             help="Path to write the canonical per-rally CSV (import-
local input)")
1173     cvat_parser.add_argument("--video-path",
1174                             help="Source video; its fps is read via ffprobe (seconds =
frame / fps)")
1175     cvat_parser.add_argument("--fps", type=float,
1176                             help="Override fps instead of probing the video")
1177     cvat_parser.add_argument("--step", type=int,
1178                             help="Frame sampling step (default: read from CVAT meta,
else inferred)")
1179     cvat_parser.add_argument("--issues",

```

9. user (2026-06-12T10:45:24.192Z)

```

55     assert parse_rate("25") == 25.0
56     assert parse_rate("0/0") is None
57     assert parse_rate("") is None
58     assert parse_rate(None) is None
59     assert parse_rate("garbage") is None
60
61
62     def test_is_vfr():
63         cfr = ProbeInfo(avg_fps=59.94006, r_fps=59.94006)
64         assert not is_vfr(cfr)
65         vfr = ProbeInfo(avg_fps=59.2, r_fps=60.0)
66         assert is_vfr(vfr)
67         # Unknown rates are treated as VFR (conservative: force the explicit flag).
68         assert is_vfr(ProbeInfo(avg_fps=None, r_fps=30.0))
69         assert is_vfr(ProbeInfo(avg_fps=30.0, r_fps=None))
70
71
72     def _info(frames=5737, fps=59.94006, dur=95.72, w=3840, h=2160):
73         return ProbeInfo(width=w, height=h, avg_fps=fps, r_fps=fps,
74                         nb_frames=frames, duration=dur)
75
76
77     def test_verify_parity_ok_and_tolerates_audio_priming():
78         src = _info()
79         # Same frames/fps; container a hair longer (AAC priming) ? still parity.
80         proxy = _info(dur=95.76, w=1920, h=1080)
81         assert verify_parity(src, proxy) == []
82
83
84     def test_verify_parity_catches_frame_count_change():

```

10. user (2026-06-12T10:45:29.043Z)

```

80     ### Context
81     The Roora pilot showed the vendor annotates in CVAT directly on our **4K / 59.94 fps

```

```

82     originals** and copes with the decode cost by sampling **every 30th frame**. The result
83     (pickleball + table-tennis review, 2026-06-10): **every rally boundary off by ~0.5 s, up
84     to ~2 s**, against the  $\pm 0.3$  s bar (Brief §4) ? all 24 boundaries sit exactly on the
85     30-frame grid (~0.5 s at 59.94 fps), so they *structurally cannot* meet spec. The same
86     grid was flagged [HIGH] in the earlier badminton round
87     (`data/roora_badminton_issues.txt`);
88     it is their standing workflow, not a one-off lapse.
89
90     **Phase B makes this untenable, not just suboptimal:** ~12?15 videos **per sport** at
91     ~30 min each (Brief §3). A 30-minute 4K/59.94 file is **>100k frames**; frame-accurate
92     annotation (frame-step 1) on 4K originals at that scale is not realistic for the vendor.
93     A lightweight **annotation proxy** (downscaled, scrub-friendly re-encode of the
94     identical
95     timeline) is therefore close to a **precondition for Phase B**, not an optimization.
96
97     The pipeline is proxy-friendly by construction: annotations join by the human-assigned
98     `video_id` (not file MD5), and `convert-cvat` recomputes all times as `frame / fps`,
99     ignoring vendor time attributes. The Brief's "do not re-encode" rule binds the *vendor*
100     (timeline + checksum identity of the file they annotate), not us ? provided downstream
101     processing always runs on the **original** file (see `PIPELINE_ROLE.md`: the indexer
102     keys
103     by MD5).
104
105     ### Decision
106
107     **We pre-process every video before vendor handoff into an annotation proxy, and the
108     generator lives in this repo as a `curate prep-annotation-proxy` subcommand** ? not a
109     standalone script, and not in the indexer.
110
111     ### Proxy contract (invariants the tool must enforce)
112
113     - **Same fps as the source ? never reduce it.** At a lower fps every CVAT frame index
114     changes meaning, and the vendor's 30-frame sampling habit at 30 fps would become a
115     **1.0 s** grid ? twice as coarse. Same frame count, no trimming, no leading padding ?
116     proxy seconds map 1:1 onto the original.
117
118     - **Downscale 4K ? 1080p** (not 720p: table-tennis ball visibility), CRF ~18?20,
119     **dense keyframes** (`-g 15`) so frame-stepping/back-scrubbing is fast, **audio kept**
120     (contact sound is the annotator's best boundary cue).
121
122     - **Verification built in:** assert proxy frame count == original frame count and
123     durations match (also surfaces VFR sources, where `frame / fps` is already unsafe).
124
125     - **Provenance:** emit/append the intake-manifest row ? `video_id`, fps, frame count,
126     **original MD5 + proxy MD5** ? so the proxy?original mapping is durable and downstream
127     processing provably targets the original.
128
129     ### Rationale (why the collector, why not a loose script)
130
131     - **This repo owns the vendor flow end-to-end** (`docs/annotation/`, intake manifest,
132     `convert-cvat` ? `validate-delivery` ? `import-local`); proxy prep is simply command
133     #0 of that existing pipeline, and it already shells out to ffprobe.
134
135     - **The ffmpeg invocation is the trivial 20%.** The durable value is the parity
136     verification + manifest provenance ? exactly what one-off scripts skip. We have
137     precedent for the failure mode: unfilled intake-manifest TODOs, eval artifacts lost in
138     `%TEMP%`, and indexer predictions once keyed under a *downscaled copy's* MD5.
139
140     - **Phase-B frequency:** ~40 runs across sessions/operators ? needs tests + CI, i.e.
141     repo-resident code.
142
143     - **Rejected ? standalone script:** acceptable stopgap for an immediate redelivery, but
144     no durable verification/provenance. **Rejected ? indexer:** it has the re-encode
145     primitive (`extract_video_chunk`) but is the product pipeline; the vendor handoff
146     workflow, manifest, and ingestion all live here.
147
148     ### Consequences / follow-ups
149
150     - **A proxy alone does not fix boundaries.** It removes the performance excuse; the
151     binding fix is the convention from the 2026-06-10 review, to be written into the
152     Phase-B brief revision: *annotate at frame step 1; place the first/last box of every

```

11. user (2026-06-12T10:46:17.877Z)

Structured output provided successfully

12. assistant (2026-06-12T10:46:23.351Z)

Verification complete ? the claim is confirmed as real with low severity.